

Programmation C

10 – Compléments sur les entrées-sorties

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique
Faculté Sciences et Ingénierie
Université de Toulouse

Licence
Semestre pair 2025-2026

 | Département
Informatique

 Faculté
sciences et
ingénierie
Université
de Toulouse

Sommaire

- 1 Introduction
- 2 Entrées-sorties sur les chaînes de caractères
- 3 Gestion des tampons d'entrée et de sortie

Sommaire

- 1 Introduction
- 2 Entrées-sorties sur les chaînes de caractères
- 3 Gestion des tampons d'entrée et de sortie

Introduction

Support complémentaire



Travaillez à partir du support complémentaire (chapitre 3).
Ici, nous ne verrons que les entrées-sorties sur les chaînes de caractères et le fonctionnement des tampons (*buffers*).

Unités standard

- l'entrée standard (`stdin`) : par défaut, le clavier ;
- la sortie standard (`stdout`) : par défaut, l'écran ;
- la sortie standard des erreurs (`stderr`) : par défaut, l'écran.

L'entrée est ouverte en lecture et les sorties sont ouvertes en écriture.

Sommaire

- 1 Introduction
- 2 Entrées-sorties sur les chaînes de caractères**
- 3 Gestion des tampons d'entrée et de sortie

Écriture dans une chaîne (1/2)

Fonction `sprintf`

La fonction `sprintf` attend un premier paramètre qui désigne une chaîne de caractères. Au lieu d'écrire sur `stdout` comme le fait `printf`, elle écrit dans la chaîne.

Exemple 1

```
char ch[100];  
float pi=3.14;  
int i=12;  
sprintf(ch,"pi=%f, _i=%d",pi,i);  
printf("' %s' \n",ch);  
produit sur stdout :  
'pi=3.140000, _i=12'
```

Écriture dans une chaîne (2/2)

Exemple 2 : construction d'un format pour la lecture d'un mot

Rappel : la fonction `scanf` avec le spécificateur de format `%s` et un gabarit permet de lire une chaîne de caractères tapée au clavier en s'arrêtant si un caractère séparateur est rencontré ou si le nombre de caractères précisé par le gabarit a été atteint.

#define NB_CAR_MAX 5 // *Nombre maximum de caractères à lire*

#define NB_CHIFFRES_MAX 3 // *Nombre maximum de chiffres du gabarit*

char Format[NB_CHIFFRES_MAX+3]; // *Format pour scanf ('%' + gabarit + 's' + '\0')*

`sprintf`(Format, "%%ds", NB_CAR_MAX); // *Écriture du format (%% pour écrire le caractère %)*

char ch[NB_CAR_MAX+1]; // *Pour stocker les chaînes*

`scanf`(Format, ch); // *Lecture de la première chaîne*

`printf`("' %s' \n", ch); // *Affichage*

`scanf`(Format, ch); // *Lecture de la deuxième chaîne*

`printf`("' %s' \n", ch); // *Affichage*

`scanf`(Format, ch); // *Lecture de la troisième chaîne*

`printf`("' %s' \n", ch); // *Affichage*

Si l'utilisateur tape :

Feuille_morte

ce fragment de programme produit sur stdout :

'Feuil'

'le'

'morte'

Lecture dans une chaîne

Fonction `sscanf`

La fonction `sscanf` attend un premier paramètre qui désigne une chaîne de caractères. Au lieu de lire sur `stdin` comme le fait `scanf`, elle lit dans la chaîne.

Exemple

```
char ch[]="12";
```

```
int i;
```

```
sscanf(ch,"%d",&i);
```

permet d'affecter l'entier 12 à la variable `i`.

La valeur de retour de `sscanf` (qui n'est pas utilisée dans cet exemple) est le nombre de valeurs mémorisées ou EOF en cas de problème (par exemple si aucune valeur n'a pu être lue en accord avec le format avant la fin de la chaîne). Dans l'exemple précédent, la valeur retournée est 1.

Sommaire

- 1 Introduction
- 2 Entrées-sorties sur les chaînes de caractères
- 3 Gestion des tampons d'entrée et de sortie

Fonctionnement du tampon d'entrée (1/3)

Les tampons

À chaque flux d'entrée et de sortie est associée une zone mémoire appelée tampon ou *buffer* qui sert d'intermédiaire entre le programme et le périphérique concerné.

Le tampon d'entrée

- Il contient les caractères tapés par l'utilisateur.
- Il fonctionne selon le principe de la file (premier arrivé = premier sorti).
- Lors de l'exécution de `scanf("%c",&car);` (ou `car=getchar();`) deux situations :
 - tampon d'entrée pas vide : la variable `car` reçoit le caractère le plus ancien se trouvant dans le tampon d'entrée et ce caractère est enlevé du tampon d'entrée (consommé) ;
 - tampon d'entrée vide : le déroulement du programme est stoppé jusqu'à ce que le tampon d'entrée reçoive un ou plusieurs caractères.
- L'ajout de caractères dans le tampon d'entrée ne se fait qu'au moment où l'utilisateur valide sa frappe avec la touche « Entrée » (`↵`) correspondant à `'\n'`.
Exemple : si l'utilisateur tape les caractères `texTe_SAisi↵`
 - le contenu du tampon d'entrée n'est modifié que lors de la frappe de `↵` ;
 - il reçoit 12 caractères.

Fonctionnement du tampon d'entrée (2/3)

Vidage du tampon d'entrée

Nécessité de vider le tampon d'entrée lors de la lecture d'un caractère, à un moment où le tampon d'entrée risque de ne pas être vide.

Exemple : la séquence suivante n'est pas correctement écrite :

```
printf("Etes-vous_d'accord_(o/n)_?\n"); /* M1 */
char rep;
rep=getchar();
while ((rep!='o') && (rep!='n'))
{
    printf("Tapez_o_ou_n:\n"); /* M2 */
    rep=getchar();
}
```

En effet, si l'utilisateur tape, par mégarde, la séquence $a \leftarrow$, en réponse au message M1, alors le message M2 s'affichera deux fois au lieu d'une, puisque le tampon d'entrée contient deux caractères (a et $\leftarrow$) différents des deux caractères autorisés (o et n). Pour remédier à ce problème, il faut procéder à un vidage conditionnel du tampon d'entrée avant l'instruction de lecture de la boucle.

Fonctionnement du tampon d'entrée (3/3)

Exemple de vidage conditionnel du tampon d'entrée

```
printf("Etes-vous_d'accord_(o/n)_?\n"); /* M1 */
char rep;
rep=getchar();
while ((rep!='o') && (rep!='n'))
{
    printf("Tapez_o_ou_n_:\n"); /* M2 */
    if (rep!='\n') while (getchar()!='\n');
    rep=getchar();
}
```

La condition `(rep!='\n')` permet de prendre en compte la possibilité que l'utilisateur ait pu taper `↵` seulement. En effet, dans ce cas, si l'on omet la condition, la boucle `while (getchar()!='\n');` serait bloquante.

Fonctionnement du tampon de sortie

Vidage du tampon de sortie

- Les fonctions d'affichage envoient les caractères non pas directement à l'écran, mais dans le tampon de sortie.
- L'affichage sur l'écran accompagné du vidage du tampon de sortie se fait dans deux cas :
 - soit quand le caractère '`\n`' doit être affiché ;
 - soit quand on utilise explicitement l'instruction `fflush(stdout)` ;

Pour éviter des confusions dues à une mauvaise synchronisation entre les entrées clavier et les affichages à l'écran, en phase de mise au point, il est conseillé de faire suivre les instructions d'affichage de l'instruction `fflush(stdout)` ; si la chaîne à afficher ne se termine pas par le caractère '`\n`' .

QCM 10.1 – 10.2

